

Truncated-normal calibration: Single predictor implementation notes

Ryan Kelly

August 19, 2026

Numbered equations from “Calibration under truncation: working notes” are referenced and implemented here.

1 The core functions

`m_tn`, `v_tn`, and `r2_map` are the direct implementations of the truncated moment equations (1)–(3) and the R^2 forward map (26).

```
m_tn <- function(mu, sigma, a, b){  
  alpha <- (a-mu)/sigma  
  beta <- (b-mu)/sigma  
  Z <- pnorm(beta)-pnorm(alpha)  
  mu + sigma*((dnorm(alpha)-dnorm(beta))/Z)  
}
```

- The truncated mean equation (1). Standardized bounds are defined as α and β , the surviving mass is defined as Z , then the mean formula is defined with `dnorm` and `pnorm` as φ and Φ .

```
v_tn <- function(mu, sigma, a, b){  
  alpha <- (a-mu)/sigma  
  beta <- (b-mu)/sigma  
  Z <- pnorm(beta)-pnorm(alpha)  
  ta <- ifelse(is.finite(alpha), alpha*dnorm(alpha), 0)  
  tb <- ifelse(is.finite(beta), beta*dnorm(beta), 0)  
  (sigma^2)*(1+((ta-tb)/Z)-((dnorm(alpha)-dnorm(beta))/Z)^2)  
}
```

- The truncated variance equation (2). At an infinite bound, the term $\alpha\varphi(\alpha)$ evaluates as $\infty \cdot 0$, which returns as `NaN`. However, because φ decays exponentially while the bound grows linearly, the true limit is 0. The `ifelse` lines directly encode that limit so when a bound is infinite, its boundary term is set to 0.

```

r2_map <- function(beta0, beta1, sigma_eps, X, a, b){
  eta <- beta0 + beta1 * X
  m <- m_tn(eta, sigma_eps, a, b)
  v <- v_tn(eta, sigma_eps, a, b)
  covhat <- mean((X - mean(X)) * (m - mean(m)))
  varX <- mean((X - mean(X))^2)
  covhat^2 / (varX * (mean(v) + mean((m - mean(m))^2)))
}

```

- The R^2 map equation (26). First, the previous `m_tn` and `v_tn` functions are used to find the conditional, truncated moments of the child.
- `covhat` is the empirical covariance between the parent values and the conditional, per-observation, truncated means of the child, $\frac{1}{n} \sum_{i=1}^n (X_i - \bar{X})(m(\eta_i, \sigma_\varepsilon) - \bar{m})$. `varX` is the empirical variance of the parent, $\frac{1}{n} \sum_{i=1}^n (X_i - \bar{X})^2$. Both are written in equation (26) as explicit sums with $1/n$, so instead of calling R's built-in `cov()` and `var()` which divide by $n - 1$, we use `mean()` to match the convention.
- Finally, the R^2_{map} equation (26) is assembled. Because the child's conditional moments can be calculated before sampling, this can be used to calibrate R^2 by using `nleqslv` to find the `beta0`, `beta1`, and `sigma_eps` required for the desired R^2 value.

2 Exogenous calibration

`calibrate_exo(mu_t, var_t, a, b)` solves the exogenous system, equation (6). Since there is no algebraic way to invert Φ , root finding via `nleqslv` is used.

Find (μ_p, σ_p) with

$$m(\mu_p, \sigma_p) = \mu_t \quad \text{and} \quad v(\mu_p, \sigma_p) = \sigma_t^2.$$

```

fn <- function(x){
  c(m_tn(x[1], x[2], a, b) - mu_t,
    v_tn(x[1], x[2], a, b) - var_t)
}
x0 <- c(mu_t, sqrt(var_t))
sol <- nleqslv(x0, fn)

```

- `fn` rewrites the system as $LHS - RHS = 0$ so solving the equations means finding the μ_p, σ_p which satisfy both equations. `nleqslv` searches for this root by calling `fn` repeatedly and using the responses to adjust toward the root.
- The starting point, `x0` = $(\mu_t, \sqrt{\sigma_t^2})$, is the solution the system would have if there were no truncation. It is the exact solution when the bounds do not bind, and a nearby one when they bind moderately, so the search starts close.
- The returned list includes `termcd`, the solver's exit code (1 = converged), and `fvec`, the remaining residuals at the final point, indicating how far each equation is from satisfied. This means that when the targets are impossible to reach the solver returns a non-converged `termcd` instead of an error. This should be checked on each use.

3 Endogenous calibration (single parent)

`calibrate_endo(mu_t, var_t, r2_t, X, a, b)` solves the endogenous system, equations (15) and (26).

Find $(\beta_0, \beta_1, \sigma_\varepsilon)$ with

$$\frac{1}{n} \sum_{i=1}^n m(\eta_i, \sigma_\varepsilon) = \mu_t, \quad \frac{1}{n} \sum_{i=1}^n v(\eta_i, \sigma_\varepsilon) + \widehat{\text{Var}}(m(\eta_i, \sigma_\varepsilon)) = \sigma_t^2, \quad R_{\text{map}}^2(\beta_0, \beta_1, \sigma_\varepsilon) = R_t^2.$$

```
fn <- function(x) {
  beta0 <- x[1]
  beta1 <- x[2]
  sig <- x[3]
  eta <- beta0 + beta1 * X
  m <- m_tn(eta, sig, a, b)
  v <- v_tn(eta, sig, a, b)
  c(mean(m) - mu_t, mean(v) + mean((m-mean(m))^2) - var_t,
    r2_map(beta0, beta1, sig, X, a, b) - r2_t)
}
vX <- mean((X - mean(X))^2)
b1_0 <- sqrt(r2_t * var_t / vX)
s_0 <- sqrt((1 - r2_t) * var_t)
b0_0 <- mu_t - b1_0 * mean(X)
sol <- nleqslv(c(b0_0, b1_0, s_0), fn)
```

- `fn` rewrites all three equations as $LHS - RHS = 0$ and uses `nleqslv` to find the $\beta_0, \beta_1, \sigma_\varepsilon$ which satisfy these equations.
- The starting values `b1_0`, `s_0`, `b0_0` are the untruncated system solved algebraically,

$$\beta_1^{(0)} = \sqrt{\frac{R_t^2 \sigma_t^2}{\widehat{\text{Var}}(X)}}, \quad \sigma_\varepsilon^{(0)} = \sqrt{(1 - R_t^2) \sigma_t^2}, \quad \beta_0^{(0)} = \mu_t - \beta_1^{(0)} \bar{X}.$$

As in the exogenous case, this starts the solver close to an answer in most cases.

- The positive square root in `b1_0` is chosen, but since β_1 enters the R^2 equation squared, a mirrored solution with negative slope exists. Starting positive selects the positive-slope root.

4 Validation design

In order to validate the above code, four categories of tests were run in `test_exo.R` and `test_endo.R`.

- **Structural-** These test exact mathematical facts such as symmetric bounds not shifting the mean ($m(0, 1, -1.5, 1.5) = 0$), the untruncated case preserving variance ($v(3, 2, -\infty, \infty) = 4$), and the one-sided reduction (the mean formula collapses to the inverse Mills ratio, $\varphi(\alpha)/(1 - \Phi(\alpha))$).
- **Independent evaluation-** These compare against reference values computed using the same formulas outside the implementation (by hand/Wolfram Alpha/Python). This ensures that the code was correctly written per the specification formulas.
- **Self-consistency-** These tests use chosen parameters to evaluate the forward map, then solve back using the calibration. This ensures the solver is behaving as expected.
- **Realized data-** These test that the analytic theory is matching data that are actually generated. The R^2 forward map is checked against a Monte Carlo study of realized R^2 , and the calibrators are checked via round trips (solve, generate, measure) to ensure they're hitting targets.

Each tolerance is sized to the largest legitimate error source of its comparison at roughly four times that source.

- **Structural:** 10^{-12} . To account for floating-point rounding.
- **Independent evaluation:** 10^{-6} . The reference values were recorded to six decimals, so this is the precision we can claim.
- **Solver recoveries:** 10^{-5} to 10^{-8} . `nleqslv` stops when the residuals in `fvec` are smaller than its internal tolerance, not always when it has found an exact root. This means the returned parameters are within a small range of the true solution, set by the solver's stopping rule. The test tolerances, then, are sized to that range. The identity test uses 10^{-8} because its start *is* the root, so the solver should converge quickly and tightly. The inversions iterate from nearby so we aim to land within 10^{-5} .
- **Stochastic tests:** $4 \times$ the sampling standard deviation of the quantity being checked. The standard error of a mean of n independent draws is:

$$\text{se}(\bar{y}) = \sqrt{\frac{\sigma^2}{n}}.$$

The standard deviation of a sample variance under approximate normality is:

$$\text{sd}(s^2) \approx \sigma^2 \sqrt{\frac{2}{n}}.$$

Since truncation makes the data lighter-tailed than normal, this tolerance is conservative.

The standard deviation of R^2 we assemble by the delta method: $\text{sd}(\hat{r}) \approx (1 - \rho^2)/\sqrt{n}$ is the classical large-sample result for the sample correlation, a first-order Taylor expansion gives $\text{sd}(g(\hat{r})) \approx |g'(\rho)| \text{sd}(\hat{r})$, and $g(r) = r^2$ gives $g'(\rho) = 2\rho$. This leaves us with:

$$\text{sd}(R^2) \approx \frac{2\rho(1 - \rho^2)}{\sqrt{n}}.$$

5 Results

Both test files pass in full. There are sixteen checks in `test_exo.R` (seed 8112026) and twenty-four in `test_endo.R` (seed 8132026). The output is reproduced verbatim below.

Exogenous Tests These tests check the moment functions against the reference values, the calibrator against known and computed answers, and a full round trip through both samplers.

```
== Moment functions vs independent reference values ==
m_tn(0,1,-1.5,1.5) [symmetric] got 0.000000 expected 0.000000 diff +0.00e+00 PASS
v_tn(0,1,-1.5,1.5) [symmetric] got 0.551524 expected 0.551524 diff +4.16e-07 PASS
m_tn(0,1,1,Inf) [one-sided] got 1.525135 expected 1.525135 diff +2.76e-07 PASS
v_tn(3,2,-Inf,Inf) [identity] got 4.000000 expected 4.000000 diff +0.00e+00 PASS
m_tn(1,0.5,0,3) [asymmetric] got 1.027556 expected 1.027556 diff +3.52e-07 PASS
v_tn(1,0.5,0,3) [asymmetric] got 0.221479 expected 0.221479 diff -1.29e-07 PASS

== calibrate_exo: analytic tests ==
identity: mu (non-binding bounds) got 3.000000 expected 3.000000 diff +0.00e+00 PASS
identity: sigma got 2.000000 expected 2.000000 diff +0.00e+00 PASS
inversion - symmetric: mu (std normal +/-1.5) got 0.000000 expected 0.000000 diff +0.00e+00 PASS
inversion - symmetric: sigma got 1.000000 expected 1.000000 diff -4.62e-08 PASS
inversion - asymmetric: mu (fwd moments of 1,0.5) got 1.000000 expected 1.000000 diff -4.38e-07 PASS
inversion - asymmetric: sigma got 0.500000 expected 0.500000 diff +9.75e-08 PASS
solver exit codes termcd = 1 1 1 (1 = converged)

== Round-trip: solve, sample, measure (targets 1, 0.5 on [0,3]) ==
solved parameters mu_p = 0.0954 sigma_p = 1.2741
inverse-CDF sampler: mean(y) got 0.998143 expected 1.000000 diff -1.86e-03 PASS
inverse-CDF sampler: var(y) got 0.499664 expected 0.500000 diff -3.36e-04 PASS
DagFlow sample_normal: mean(y) got 0.999724 expected 1.000000 diff -2.76e-04 PASS
DagFlow sample_normal: var(y) got 0.500327 expected 0.500000 diff +3.27e-04 PASS
```

Endogenous Tests These tests verify the R^2 forward map against the R^2 simulation, the calibrator against the identity and inversion checks, and round trips at $n = 5000$ and $n = 50000$.

```
== r2_map vs Aug 4 simulation table (tol = 4*mc_se ~ 0.006) ==
latent 0.30, bounds +/-Inf got 0.300090 expected 0.300000 diff +8.98e-05 PASS
latent 0.30, bounds +/-3 got 0.229367 expected 0.229800 diff -4.33e-04 PASS
latent 0.30, bounds +/-2.5 got 0.194933 expected 0.193900 diff +1.03e-03 PASS
latent 0.30, bounds +/-2 got 0.151841 expected 0.150400 diff +1.44e-03 PASS
latent 0.30, bounds +/-1.5 got 0.102455 expected 0.103300 diff -8.45e-04 PASS
latent 0.30, bounds +/-1.25 got 0.077177 expected 0.078100 diff -9.23e-04 PASS
latent 0.30, bounds +/-1 got 0.053077 expected 0.053300 diff -2.23e-04 PASS
latent 0.60, bounds +/-Inf got 0.600103 expected 0.600000 diff +1.03e-04 PASS
latent 0.60, bounds +/-3 got 0.582713 expected 0.582100 diff +6.13e-04 PASS
latent 0.60, bounds +/-2.5 got 0.560739 expected 0.562600 diff -1.86e-03 PASS
latent 0.60, bounds +/-2 got 0.520193 expected 0.521900 diff -1.71e-03 PASS
latent 0.60, bounds +/-1.5 got 0.450176 expected 0.450000 diff +1.76e-04 PASS
latent 0.60, bounds +/-1.25 got 0.398297 expected 0.398100 diff +1.97e-04 PASS
latent 0.60, bounds +/-1 got 0.330446 expected 0.329700 diff +7.46e-04 PASS

== calibrate_endo: analytic tests ==
identity - non-binding bounds: sigma_eps got 0.836660 expected 0.836660 diff +0.00e+00 PASS
identity: full solution beta0 = -0.00671 beta1 = 0.54933 termcd = 1
inversion: beta0 got 0.300000 expected 0.300000 diff +1.31e-08 PASS
inversion: beta1 got 0.900000 expected 0.900000 diff +1.19e-08 PASS
inversion: sigma_eps got 0.700000 expected 0.700000 diff +1.14e-08 PASS

== Round-trip via sample_normal (targets 0, 0.5, 0.30 on +/-1.5) ==
-- n = 5000 (single-dataset) --
realized mean(y) got 0.015028 expected 0.000000 diff +1.50e-02 PASS
realized var(y) got 0.499296 expected 0.500000 diff -7.04e-04 PASS
```

realized R ²	got	0.282665	expected	0.300000	diff	-1.73e-02	PASS
-- n = 50000 (single-dataset) --							
realized mean(y)	got	0.004896	expected	0.000000	diff	+4.90e-03	PASS
realized var(y)	got	0.504583	expected	0.500000	diff	+4.58e-03	PASS
realized R ²	got	0.294745	expected	0.300000	diff	-5.25e-03	PASS

6 Observed behaviors and future work

Observation 1: infeasible targets stall the solver. An early test of the system before the above test files (performed Aug 7, not in the repository):

- Asked: target variance 1.0 on bounds $[-1, 1]$.
- Returned: `termcd = 3` (stalled, not converged), σ inflated to ~ 826 , and residuals $\approx (0, -0.667)$. The mean equation was satisfied while the variance equation stuck at a floor, and the returned values gave no error on their own.

I've begun investigating the $\sigma \rightarrow \infty$ limit in the system to explain this result.

Observation 2: a numerical failure.

- Asked: endogenous targets $(0, 1.0, 0.30)$ on ± 1.5 .
- Returned: parameters that crashed `sample_normal`.

My thought is that η_i far outside the bounds drives the CDF values together in floating point, and `qnorm` returns non-finite draws but this hasn't been verified. This means the calibrator can return parameters that break the sampler numerically. Guards for this may need to be put into place.